



LABORATORY RECORD

submitted to

**ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
LABORATORY**

(CSE531P)

by

Name	Deepak Reddy
Register No	2460356
Class/Section	5 BTCS C

FACULTY IN-CHARGE: DR SHARON ROJI

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF ENGINEERING AND TECHNOLOGY
CHRIST (DEEMED TO BE UNIVERSITY)

Academic Year: 2026-27



CHRIST
(DEEMED TO BE UNIVERSITY)
BANGALORE | DELHI NCR | PUNE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF ENGINEERING AND TECHNOLOGY

CERTIFICATE

This is to certify that Mr Deepak Reddy, bearing Register No. 2460356, has satisfactorily completed the Laboratory record in the laboratory course, CSE531P - ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING prescribed by the Department of Computer Science and Engineering, CHRIST (Deemed to be University) for the V semester B.Tech. program during the academic year 2026-27.

Dr Sharon Roji
Faculty In-charge

Dr. Balamurgan M
Head of the Department

Date:

Place: Bangalore



CHRIST
(DEEMED TO BE UNIVERSITY)
BANGALORE | DELHI NCR | PUNE

Vision

"Excellence and Service"

Mission

"CHRIST (Deemed to be University) is a nurturing ground for an individual's holistic development to make effective contribution to the society in a dynamic environment."

Core Values

*Faith in God
Moral Uprightness
Love of Fellow Beings
Social Responsibility
Pursuit of Excellence*



CHRIST
(DEEMED TO BE UNIVERSITY)
BANGALORE | DELHI NCR | PUNE

Department Vision

"To fortify Ethical Computational Excellence"

Department Mission

- M1:** Imparts core and contemporary knowledge in the areas of Computation and Information Technology.
- M2:** Promotes the culture of research and facilitates higher studies and lifelong learning.
- M3:** Acquaints the students with the latest industrial practices, team building and entrepreneurship.
- M4:** Sensitizes the students to serve for environmental, social and ethical needs of society.

Program Educational Objectives (PEOs)

- PEO1: Professional Acumen:** Understand, analyze and design solutions with professional competency for real-world problems.
- PEO2: Critical Analysis:** Develop software/embedded solutions for the requirements, based on critical analysis and research.
- PEO3: Collaborative Practice:** Work effectively in an interdisciplinary/multicultural environment.
- PEO4: Ethical Responsibility:** Demonstrate professional ethics, social awareness and lifelong learning responsibilities.

Program Outcomes (POs)

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization to develop the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions.

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

Program Specific Outcomes (PSOs)

PSO1: To gain knowledge in the basics of Artificial Intelligence.

PSO2: To analyze the applicability of searching algorithms in real time scenarios.

PSO3: To acquire knowledge on knowledge representation and Learning.

PSO4: To enable students to apply machine learning techniques for data analysis, pattern recognition, and predictive modelling.

PSO5: To foster problem-solving skills by implementing AI and ML algorithms in real-world applications.

INDEX

Experiment Number	Experiment Name	Date	Marks	Signature
01	Basic Python Programs	08/06/2026		
02	Water Jug Problem Using BFS and DFS	15/06/2026		
03(A)	Greedy Best First Search	22/06/2026		
03(B)	A Star Search	29/06/2026		
04	Hill Climbing Algorithm	06/07/2026		
05(A)	Minimax Algorithm for Tic-Tac-Toe	13/07/2026		
05(B)	Alpha-Beta Pruning for Tic-Tac-Toe	13/07/2026		
06	Cryptarithmic Puzzle	21/07/2026		

Date: 08/06/2026

Experiment - 01

Basic Python Programs

Aim: To implement basic Python programs using control statements, loops, functions, strings, lists, mathematical operations, and validation techniques.

A. Numbers divisible by 7 but not multiples of 5

Algorithm:

- Step 1 : Start the program.
- Step 2 : Read the starting and ending numbers from the user.
- Step 3 : Create an empty list to store the required numbers.
- Step 4 : Visit every number from the starting value to the ending value.
- Step 5 : Check whether the number is divisible by 7.
- Step 6 : Check that the same number is not divisible by 5.
- Step 7 : Store every number satisfying both conditions.
- Step 8 : Print the stored numbers as a comma-separated sequence and end the program.

Code:

```
1 start = int(input("Enter starting number: "))
2 end = int(input("Enter ending number: "))
3
4 result = []
5 for number in range(start, end + 1):
6     if number % 7 == 0 and number % 5 != 0:
7         result.append(str(number))
8
9 print(",".join(result))
```

Sample Output:

```
Enter starting number: 2000
Enter ending number: 3200
2002,2009,2016,2023,2037,2044,2051,2058
2072,2079,2086,2093,2107,2114,2121,2128
2142,2149,2156,2163,2177,2184,2191,2198
2212,2219,2226,2233,2247,2254,2261,2268
2282,2289,2296,2303,2317,2324,2331,2338
2352,2359,2366,2373,2387,2394,2401,2408
2422,2429,2436,2443,2457,2464,2471,2478
```



```
2492,2499,2506,2513,2527,2534,2541,2548
2562,2569,2576,2583,2597,2604,2611,2618
2632,2639,2646,2653,2667,2674,2681,2688
2702,2709,2716,2723,2737,2744,2751,2758
2772,2779,2786,2793,2807,2814,2821,2828
2842,2849,2856,2863,2877,2884,2891,2898
2912,2919,2926,2933,2947,2954,2961,2968
2982,2989,2996,3003,3017,3024,3031,3038
3052,3059,3066,3073,3087,3094,3101,3108
3122,3129,3136,3143,3157,3164,3171,3178
3192,3199
```

B. Factorial of a number

Algorithm:

- Step 1 : Start the program.
- Step 2 : Read a non-negative integer from the user.
- Step 3 : Initialize the factorial value to 1.
- Step 4 : Repeat from 1 up to the entered number.
- Step 5 : Multiply the factorial value by the current number in each repetition.
- Step 6 : Print the final factorial value.
- Step 7 : End the program.

Code:

```
1 def factorial(number):
2     fact = 1
3     for value in range(1, number + 1):
4         fact *= value
5     return fact
6
7 number = int(input("Enter a number: "))
8 print("Factorial of", number, "=", factorial(number))
```

Sample Output:

```
Enter a number: 5
Factorial of 5 = 120
```

C. Formula calculation

Algorithm:

- Step 1 : Start the program.
- Step 2 : Read C and H values from the user.

Step 3 : Read comma-separated values of D from the user.

Step 4 : Split the input into individual values.

Step 5 : Convert each value of D into an integer.

Step 6 : Calculate $Q = \sqrt{(2CD)/H}$ for each value.

Step 7 : Round each calculated value of Q .

Step 8 : Store and print the answers as a comma-separated sequence.

Step 9 : End the program.

Code:

```
1 import math
2
3 c = int(input("Enter C value: "))
4 h = int(input("Enter H value: "))
5 d_values = input("Enter comma-separated D values: ").split(',')
6 result = []
7
8 for value in d_values:
9     d = int(value)
10    q = round(math.sqrt((2 * c * d) / h))
11    result.append(str(q))
12
13 print(",".join(result))
```

Sample Output:

```
Enter C value: 50  Enter H value: 30
Enter comma-separated D values: 100,150,180
18,22,24
```

D. Two-dimensional array generation

Algorithm:

Step 1 : Start the program.

Step 2 : Read the number of rows and columns from the user.

Step 3 : Create an empty matrix.

Step 4 : Repeat for every row index i .

Step 5 : Create an empty row for the current row index.

Step 6 : Repeat for every column index j .

Step 7 : Calculate the element as $i \times j$ and add it to the row.

Step 8 : Add the completed row to the matrix and print the matrix.

Step 9 : End the program.

Code:

```
1 rows = int(input("Enter number of rows: "))
2 columns = int(input("Enter number of columns: "))
3
4 matrix = []
5 for i in range(rows):
6     row = []
7     for j in range(columns):
8         row.append(i * j)
9     matrix.append(row)
10
11 print(matrix)
```

Sample Output:

```
Enter number of rows: 3
Enter number of columns: 5
[[0, 0, 0, 0, 0], [0, 1, 2, 3, 4], [0, 2, 4, 6, 8]]
```

E. Count letters and digits

Algorithm:

Step 1 : Start the program.

Step 2 : Read a sentence from the user.

Step 3 : Initialize the letter count and digit count to zero.

Step 4 : Examine each character in the sentence.

Step 5 : If the character is a letter, increase the letter count.

Step 6 : If the character is a digit, increase the digit count.

Step 7 : Print both counts.

Step 8 : End the program.

Code:

```
1 sentence = input("Enter a sentence: ")
2 letters = 0
3 digits = 0
4
5 for char in sentence:
6     if char.isalpha():
7         letters += 1
8     elif char.isdigit():
9         digits += 1
10
```

```
11 print("LETTERS", letters)
12 print("DIGITS", digits)
```

Sample Output:

```
Enter a sentence: hello world! 123
LETTERS 10
DIGITS 3
```

F. Count uppercase and lowercase letters

Algorithm:

- Step 1 : Start the program.
- Step 2 : Read a sentence from the user.
- Step 3 : Initialize uppercase and lowercase counters to zero.
- Step 4 : Examine each character in the sentence.
- Step 5 : If the character is uppercase, increase the uppercase counter.
- Step 6 : If the character is lowercase, increase the lowercase counter.
- Step 7 : Print the uppercase and lowercase counts.
- Step 8 : End the program.

Code:

```
1 sentence = input("Enter a sentence: ")
2 upper_count = 0
3 lower_count = 0
4
5 for char in sentence:
6     if char.isupper():
7         upper_count += 1
8     elif char.islower():
9         lower_count += 1
10
11 print("UPPER CASE", upper_count)
12 print("LOWER CASE", lower_count)
```

Sample Output:

```
Enter a sentence: Hello World
UPPER CASE 2
LOWER CASE 8
```

G. Password validation

Algorithm:

Step 1 : Start the program.

Step 2 : Read comma-separated passwords from the user.

Step 3 : Split the input into individual passwords.

Step 4 : Check that each password contains 6 to 12 characters.

Step 5 : Check for at least one lowercase letter.

Step 6 : Check for at least one uppercase letter.

Step 7 : Check for at least one digit.

Step 8 : Check for at least one of the special characters \$, #, or @.

Step 9 : Store and print only the passwords satisfying every condition.

Step 10 : End the program.

Code:

```
1 import re
2
3 passwords = input("Enter comma-separated passwords: ").split(',')
4 valid_passwords = []
5
6 for password in passwords:
7     if (6 <= len(password) <= 12 and
8         re.search(r"[a-z]", password) and
9         re.search(r"[A-Z]", password) and
10        re.search(r"[0-9]", password) and
11        re.search(r"[$#@]", password)):
12        valid_passwords.append(password)
13
14 print(",".join(valid_passwords))
```

Sample Output:

```
Enter comma-separated passwords: ABd1234@1,a F1#,2w3E*,2We3345
ABd1234@1
```

H. Capitalize input lines

Algorithm:

Step 1 : Start the program.

Step 2 : Read the number of lines to be entered.

Step 3 : Create an empty list for the lines.

Step 4 : Read each line from the user.

Step 5 : Store every entered line in the list.

Step 6 : Convert each stored line to uppercase.

Step 7 : Print the uppercase lines.

Step 8 : End the program.

Code:

```
1 number_of_lines = int(input("Enter number of lines: "))
2 lines = []
3
4 for i in range(number_of_lines):
5     lines.append(input("Enter line: "))
6
7 for line in lines:
8     print(line.upper())
```

Sample Output:

```
Enter number of lines: 2
Enter line: Hello world
Enter line: Practice makes perfect
HELLO WORLD
PRACTICE MAKES PERFECT
```

I. Remove duplicate words and sort

Algorithm:

Step 1 : Start the program.

Step 2 : Read a sentence containing whitespace-separated words.

Step 3 : Split the sentence into individual words.

Step 4 : Remove duplicate words by converting the collection to a set.

Step 5 : Sort the unique words alphabetically.

Step 6 : Print the sorted words separated by spaces.

Step 7 : End the program.

Code:

```
1 sentence = input("Enter whitespace-separated words: ")
2 words = sentence.split()
3 unique_words = sorted(set(words))
4 print(" ".join(unique_words))
```

Sample Output:

```
Enter whitespace-separated words: hello world and practice makes perfect and
hello world again
again and hello makes perfect practice world
```

J. Binary numbers divisible by 5

Algorithm:

Step 1 : Start the program.

Step 2 : Read comma-separated four-digit binary numbers.

Step 3 : Split the input into individual binary numbers.

Step 4 : Convert each binary number to its decimal value.

Step 5 : Check whether the decimal value is divisible by 5.

Step 6 : Store every binary number that satisfies the condition.

Step 7 : Print the stored binary numbers as a comma-separated sequence.

Step 8 : End the program.

Code:

```
1 binary_numbers = input("Enter comma-separated 4-digit binary numbers: ").split(',')
2 result = []
3
4 for number in binary_numbers:
5     if int(number, 2) % 5 == 0:
6         result.append(number)
7
8 print(",".join(result))
```

Sample Output:

```
Enter comma-separated 4-digit binary numbers: 0100,0011,1010,1001
1010
```

Result: All basic Python programs were successfully implemented and verified with user input and sample outputs.

Date: 15/06/2026

Experiment - 02

Water Jug Problem Using BFS and DFS

Aim: To solve the water jug problem using Breadth First Search and Depth First Search strategies.

Algorithm:

- Step 1 : Start the program and read both jug capacities and the target amount.
- Step 2 : Represent a state as the current amount of water in both jugs.
- Step 3 : Generate successor states by filling, emptying, or pouring between the jugs.
- Step 4 : For BFS, place the initial state (0,0) in a queue.
- Step 5 : Remove states from the front of the queue and ignore previously visited states.
- Step 6 : Add each valid unvisited successor state to the queue.
- Step 7 : Stop BFS when either jug contains the target amount and save the path.
- Step 8 : For DFS, place the initial state in a stack.
- Step 9 : Remove states from the top of the stack and add unvisited successors.
- Step 10 : Stop DFS when either jug contains the target amount.
- Step 11 : Print the BFS and DFS solution paths and end the program.

Code:

```
1 def get_next_states(x, y, jug1, jug2):
2     states = [(jug1, y), (x, jug2), (0, y), (x, 0)]
3
4     transfer = min(x, jug2 - y)
5     states.append((x - transfer, y + transfer))
6
7     transfer = min(y, jug1 - x)
8     states.append((x + transfer, y - transfer))
9
10    return states
11
12
13 def bfs(jug1, jug2, target):
14     queue = [(0, 0), []]
15     visited = set()
16
17     while queue:
18         current, path = queue.pop(0)
19         if current in visited:
```



```
20         continue
21     visited.add(current)
22     path = path + [current]
23
24     x, y = current
25     if x == target or y == target:
26         return path
27
28     for state in get_next_states(x, y, jug1, jug2):
29         if state not in visited:
30             queue.append((state, path))
31     return None
32
33
34 def dfs(jug1, jug2, target):
35     stack = [(0, 0), []]
36     visited = set()
37
38     while stack:
39         current, path = stack.pop()
40         if current in visited:
41             continue
42         visited.add(current)
43         path = path + [current]
44
45         x, y = current
46         if x == target or y == target:
47             return path
48
49         for state in get_next_states(x, y, jug1, jug2):
50             if state not in visited:
51                 stack.append((state, path))
52     return None
53
54
55 jug1 = int(input("Enter capacity of jug 1: "))
56 jug2 = int(input("Enter capacity of jug 2: "))
57 target = int(input("Enter target amount: "))
58
59 print("BFS Solution:")
60 bfs_path = bfs(jug1, jug2, target)
61 for state in bfs_path:
62     print(state)
63
64 print("\nDFS Solution:")
65 dfs_path = dfs(jug1, jug2, target)
66 for state in dfs_path:
```

67

```
print(state)
```

Sample Output:

```
Enter capacity of jug 1: 3
Enter capacity of jug 2: 5
Enter target amount: 4
BFS Solution:
(0, 0)
(0, 5)
(3, 2)
(0, 2)
(2, 0)
(2, 5)
(3, 4)

DFS Solution:
(0, 0)
(0, 5)
(3, 2)
(0, 2)
(2, 0)
(2, 5)
(3, 4)
```

Result: The water jug problem was solved successfully using BFS and DFS, and a state containing the target amount was obtained.

Date: 22/06/2026

Experiment - 03(A)

Greedy Best First Search

Aim: To find a path from a start node to a goal node using Greedy Best First Search with heuristic values.

Algorithm:

- Step 1 : Start the program.
- Step 2 : Read the graph edges, heuristic values, start node, and goal node.
- Step 3 : Insert the start node into a priority queue using its heuristic value.
- Step 4 : Create an empty visited set and parent mapping.
- Step 5 : Remove the node having the smallest heuristic value.
- Step 6 : If it is the goal node, reconstruct the path using the parent mapping.
- Step 7 : Otherwise, mark the current node as visited.
- Step 8 : Examine every unvisited neighbor of the current node.
- Step 9 : Insert each neighbor into the priority queue using its heuristic value.
- Step 10 : Record the current node as the neighbor's parent when required.
- Step 11 : Print the discovered path or a failure message and end the program.

Code:

```
1 import heapq
2
3
4 def greedy_best_first_search(start, goal, graph, heuristic):
5     open_set = []
6     heapq.heappush(open_set, (heuristic[start], start))
7     came_from = {}
8     visited = set()
9
10    while open_set:
11        _, current = heapq.heappop(open_set)
12
13        if current == goal:
14            path = [current]
15            while current in came_from:
16                current = came_from[current]
17            path.append(current)
18            return path[::-1]
19
```

```
20         visited.add(current)
21         for neighbor in graph.get(current, {}):
22             if neighbor not in visited:
23                 heapq.heappush(open_set, (heuristic[neighbor], neighbor))
24                 if neighbor not in came_from:
25                     came_from[neighbor] = current
26         return None
27
28
29 edges = int(input("Enter number of edges: "))
30 graph = {}
31 print("Enter edges in the format: from to cost")
32 for _ in range(edges):
33     source, destination, cost = input().split()
34     graph.setdefault(source, {})[destination] = int(cost)
35     graph.setdefault(destination, {})
36
37 heuristic = {}
38 entries = int(input("Enter number of heuristic entries: "))
39 print("Enter heuristic values in the format: node value")
40 for _ in range(entries):
41     node, value = input().split()
42     heuristic[node] = int(value)
43
44 start = input("Enter start node: ")
45 goal = input("Enter goal node: ")
46 path = greedy_best_first_search(start, goal, graph, heuristic)
47
48 if path:
49     print("Path found:", " -> ".join(path))
50 else:
51     print("No path found.")
```

Sample Output:

```
Enter number of edges: 5
Enter edges in the format: from to cost
A B 2
A E 3
B C 1
B G 9
E D 6
Enter number of heuristic entries: 6
Enter heuristic values in the format: node value
A 11
B 6
C 99
E 7
```

```
D 1
G 0
Enter start node: A
Enter goal node: G
Path found: A -> B -> G
```

Result: Greedy Best First Search was successfully implemented using heuristic values to guide the search.

Date: 29/06/2026

Experiment - 03(B)

A Star Search

Aim: To find the shortest path from a start node to a goal node using the A star search algorithm.

Algorithm:

- Step 1 : Start the program.
- Step 2 : Read the weighted graph, heuristic values, start node, and goal node.
- Step 3 : Set the start node cost to zero and insert it into a priority queue.
- Step 4 : Calculate priority using $f(n) = g(n) + h(n)$.
- Step 5 : Remove the node with the smallest $f(n)$ value.
- Step 6 : Skip the node if it has already been visited.
- Step 7 : If the current node is the goal, stop the search.
- Step 8 : For each neighbor, calculate the tentative path cost $g(n)$.
- Step 9 : If the new cost is lower, update its cost and parent.
- Step 10 : Insert the neighbor with its updated $f(n)$ value.
- Step 11 : Reconstruct the path from the goal using the parent mapping.
- Step 12 : Print the visited nodes and shortest path, then end the program.

Code:

```
1 from queue import PriorityQueue
2
3
4 def a_star_search(graph, heuristics, start, goal):
5     open_set = PriorityQueue()
6     open_set.put((heuristics[start], 0, start))
7     came_from = {start: None}
8     g_score = {start: 0}
9     visited = set()
10
11     while not open_set.empty():
12         _, cost_so_far, current = open_set.get()
13         if current in visited:
14             continue
15
16         print(f"Visiting Node: {current}")
17         visited.add(current)
```

```
18
19     if current == goal:
20         break
21
22     for neighbor, weight in graph.get(current, []):
23         tentative_g = g_score[current] + weight
24         if neighbor not in g_score or tentative_g < g_score[neighbor]:
25             g_score[neighbor] = tentative_g
26             f_score = tentative_g + heuristics[neighbor]
27             open_set.put((f_score, tentative_g, neighbor))
28             came_from[neighbor] = current
29
30     path = []
31     node = goal
32     while node is not None:
33         path.append(node)
34         node = came_from.get(node)
35     return path[::-1]
36
37
38 nodes = int(input("Enter number of nodes: "))
39 graph = {}
40 print("Enter connections with weights:")
41 for _ in range(nodes):
42     node = input("Node name: ").strip()
43     edges = input(f"Enter neighbors and weights of {node} (e.g. B:2,C:3): ").split(',')
44     graph[node] = []
45     for edge in edges:
46         if edge.strip():
47             neighbor, weight = edge.split(':')
48             graph[node].append((neighbor.strip(), int(weight.strip())))
49
50 heuristics = {}
51 print("Enter heuristic values:")
52 for node in graph:
53     heuristics[node] = int(input(f"Heuristic value for {node}: "))
54
55 start = input("Enter start node: ").strip()
56 goal = input("Enter goal node: ").strip()
57 print("--- A* Search ---")
58 path = a_star_search(graph, heuristics, start, goal)
59 print("Path from start to goal:", path)
```

Sample Output:

```
Enter number of nodes: 6
Enter connections with weights:
Node name: A
```

```
Enter neighbors and weights of A (e.g. B:2,C:3): B:2,E:3
Node name: B
Enter neighbors and weights of B (e.g. B:2,C:3): C:1,G:9
Node name: C
Enter neighbors and weights of C (e.g. B:2,C:3):
Node name: E
Enter neighbors and weights of E (e.g. B:2,C:3): D:6
Node name: D
Enter neighbors and weights of D (e.g. B:2,C:3): G:1
Node name: G
Enter neighbors and weights of G (e.g. B:2,C:3):
Enter heuristic values:
Heuristic value for A: 11
Heuristic value for B: 6
Heuristic value for C: 99
Heuristic value for E: 7
Heuristic value for D: 1
Heuristic value for G: 0
Enter start node: A
Enter goal node: G
--- A* Search ---
Visiting Node: A
Visiting Node: B
Visiting Node: E
Visiting Node: D
Visiting Node: G
Path from start to goal: ['A', 'E', 'D', 'G']
```

Result: A star search was successfully implemented and the shortest path from the start node to the goal node was obtained.

Date: 06/07/2026

Experiment - 04

Hill Climbing Algorithm

Aim: To develop a search strategy to determine the peak element in an array and find the square root of the peak element.

Algorithm:

- Step 1 : Start the program.
- Step 2 : Read the array elements from the user.
- Step 3 : If the array is empty, display an appropriate message and stop.
- Step 4 : Select the first array element as the current element.
- Step 5 : Compare the current element with the next element.
- Step 6 : Move to the next element while it has a greater value.
- Step 7 : Stop when the next element is smaller or the array ends.
- Step 8 : Treat the current element as the local maximum or peak.
- Step 9 : If the peak is non-negative, calculate and print its square root.
- Step 10 : Print the peak value and end the program.

Code:

```
1 import math
2
3
4 def hill_climb_peak(arr):
5     if not arr:
6         return None
7
8     current = 0
9     while current < len(arr) - 1 and arr[current] < arr[current + 1]:
10         current += 1
11     return arr[current]
12
13
14 arr = list(map(float, input("Enter array elements separated by space: ").split()))
15 peak = hill_climb_peak(arr)
16
17 if peak is not None:
18     print("Peak Element (Local Maxima):", peak)
19     if peak >= 0:
```

```
20         print("Square Root of Peak Element:", round(math.sqrt(peak), 2))
21     else:
22         print("Square root not possible for negative value.")
23 else:
24     print("Array is empty.")
```

Sample Output:

```
Enter array elements separated by space: 8 6 2 9 5 6
Peak Element (Local Maxima): 8.0
Square Root of Peak Element: 2.83
```

Result: The hill climbing algorithm was successfully implemented to find the peak element and calculate its square root.

Date: 13/07/2026

Experiment - 05(A)

Minimax Algorithm for Tic-Tac-Toe

Aim: To implement the Minimax algorithm for a Tic-Tac-Toe game and enable the computer to select the best possible move.

Algorithm:

- Step 1 : Start the program and initialize an empty Tic-Tac-Toe board.
- Step 2 : Display the board and read a valid move from the human player.
- Step 3 : Place the human player's symbol X in the selected position.
- Step 4 : Check whether the human player has won or the board is full.
- Step 5 : Generate every possible move available to the computer player O.
- Step 6 : Apply Minimax recursively to evaluate each possible game state.
- Step 7 : Assign 1 for a computer win, -1 for a human win, and 0 for a draw.
- Step 8 : Choose the move having the highest Minimax score.
- Step 9 : Place O in the selected position and display the updated board.
- Step 10 : Repeat the turns until a player wins or the game ends in a draw.
- Step 11 : Display the result and end the program.

Code:

```
1 import math
2
3 board = [' ' for _ in range(9)]
4
5
6 def print_board():
7     for row in range(3):
8         print("|".join(board[row * 3:(row + 1) * 3]))
9         if row < 2:
10            print("-----")
11
12
13 def check_winner(current_board, player):
14     winning_positions = [
15         (0, 1, 2), (3, 4, 5), (6, 7, 8),
16         (0, 3, 6), (1, 4, 7), (2, 5, 8),
17         (0, 4, 8), (2, 4, 6)
```

```
18     ]
19     return any(
20         current_board[i] == current_board[j] == current_board[k] == player
21         for i, j, k in winning_positions
22     )
23
24
25 def is_full(current_board):
26     return ' ' not in current_board
27
28
29 def minimax(current_board, is_maximizing):
30     if check_winner(current_board, 'O'):
31         return 1
32     if check_winner(current_board, 'X'):
33         return -1
34     if is_full(current_board):
35         return 0
36
37     if is_maximizing:
38         best_score = -math.inf
39         for position in range(9):
40             if current_board[position] == ' ':
41                 current_board[position] = 'O'
42                 score = minimax(current_board, False)
43                 current_board[position] = ' '
44                 best_score = max(best_score, score)
45         return best_score
46
47     best_score = math.inf
48     for position in range(9):
49         if current_board[position] == ' ':
50             current_board[position] = 'X'
51             score = minimax(current_board, True)
52             current_board[position] = ' '
53             best_score = min(best_score, score)
54     return best_score
55
56
57 def best_move():
58     selected_move = -1
59     best_score = -math.inf
60
61     for position in range(9):
62         if board[position] == ' ':
63             board[position] = 'O'
64             score = minimax(board, False)
```

```
65         board[position] = ' '
66
67         if score > best_score:
68             best_score = score
69             selected_move = position
70
71     return selected_move
72
73
74 def play_game():
75     print("Tic-Tac-Toe using Minimax")
76     print_board()
77
78     while True:
79         while True:
80             try:
81                 user_move = int(input("Your move (0-8): "))
82                 if user_move < 0 or user_move > 8:
83                     print("Please enter a number between 0 and 8.")
84                 elif board[user_move] != ' ':
85                     print("That spot is already taken. Try another one.")
86                 else:
87                     break
88             except ValueError:
89                 print("Please enter a valid number.")
90
91     board[user_move] = 'X'
92     print_board()
93
94     if check_winner(board, 'X'):
95         print("You win!")
96         break
97     if is_full(board):
98         print("Draw!")
99         break
100
101     computer_move = best_move()
102     board[computer_move] = 'O'
103     print("AI moved:")
104     print_board()
105
106     if check_winner(board, 'O'):
107         print("AI wins!")
108         break
109     if is_full(board):
110         print("Draw!")
111         break
```

```
112  
113  
114 play_game()
```

Sample Output:

```
Tic-Tac-Toe using Minimax  
| |  
-----  
| |  
-----  
| |  
Your move (0-8): 2  
| |X  
-----  
| |  
-----  
| |  
AI moved:  
| |X  
-----  
|O|  
-----  
| |  
Your move (0-8): 3  
| |X  
-----  
X|O|  
-----  
| |  
AI moved:  
O| |X  
-----  
X|O|  
-----  
| |  
Your move (0-8): 5  
O| |X  
-----  
X|O|X  
-----  
| |  
AI moved:  
O| |X  
-----  
X|O|X  
-----  
| |O
```

AI wins!

Result: The Minimax algorithm was successfully implemented for Tic-Tac-Toe, enabling the computer to evaluate possible states and select an optimal move.

Date: 13/07/2026

Experiment - 05(B)

Alpha-Beta Pruning for Tic-Tac-Toe

Aim: To implement Alpha-Beta Pruning for a Tic-Tac-Toe game and reduce the number of game states evaluated by the Minimax search.

Algorithm:

- Step 1 : Start the program and initialize an empty Tic-Tac-Toe board.
- Step 2 : Display the board and obtain a valid move from the human player.
- Step 3 : Place X in the selected position and check for a win or draw.
- Step 4 : Generate each available move for the computer player O.
- Step 5 : Begin the recursive search with alpha as negative infinity and beta as positive infinity.
- Step 6 : Evaluate terminal states as 1 for an O win, -1 for an X win, and 0 for a draw.
- Step 7 : Update alpha with the best maximizing score found for O.
- Step 8 : Update beta with the best minimizing score found for X.
- Step 9 : Stop exploring a branch whenever beta is less than or equal to alpha.
- Step 10 : Select and play the computer move having the highest score.
- Step 11 : Repeat until a player wins or the board is full, then display the result and end.

Code:

```
1 import math
2
3 board = [' ' for _ in range(9)]
4
5
6 def print_board():
7     for row in range(3):
8         print("|".join(board[row * 3:(row + 1) * 3]))
9         if row < 2:
10            print("-----")
11
12
13 def check_winner(current_board, player):
14     winning_positions = [
15         (0, 1, 2), (3, 4, 5), (6, 7, 8),
16         (0, 3, 6), (1, 4, 7), (2, 5, 8),
17         (0, 4, 8), (2, 4, 6)
```



```
18 ]
19 return any(
20     current_board[i] == current_board[j] == current_board[k] == player
21     for i, j, k in winning_positions
22 )
23
24
25 def is_full(current_board):
26     return ' ' not in current_board
27
28
29 def alpha_beta(current_board, alpha, beta, is_maximizing):
30     if check_winner(current_board, 'O'):
31         return 1
32     if check_winner(current_board, 'X'):
33         return -1
34     if is_full(current_board):
35         return 0
36
37     if is_maximizing:
38         best_score = -math.inf
39         for position in range(9):
40             if current_board[position] == ' ':
41                 current_board[position] = 'O'
42                 score = alpha_beta(current_board, alpha, beta, False)
43                 current_board[position] = ' '
44                 best_score = max(best_score, score)
45                 alpha = max(alpha, best_score)
46                 if beta <= alpha:
47                     break
48         return best_score
49
50     best_score = math.inf
51     for position in range(9):
52         if current_board[position] == ' ':
53             current_board[position] = 'X'
54             score = alpha_beta(current_board, alpha, beta, True)
55             current_board[position] = ' '
56             best_score = min(best_score, score)
57             beta = min(beta, best_score)
58             if beta <= alpha:
59                 break
60     return best_score
61
62
63 def best_move():
64     selected_move = -1
```

```
65     best_score = -math.inf
66
67     for position in range(9):
68         if board[position] == ' ':
69             board[position] = 'O'
70             score = alpha_beta(board, -math.inf, math.inf, False)
71             board[position] = ' '
72
73             if score > best_score:
74                 best_score = score
75                 selected_move = position
76
77     return selected_move
78
79
80 def play_game():
81     print("Tic-Tac-Toe (AI using Alpha-Beta Pruning)")
82     print_board()
83
84     while True:
85         while True:
86             try:
87                 user_move = int(input("Your move (0-8): "))
88                 if user_move < 0 or user_move > 8:
89                     print("Please enter a number between 0 and 8.")
90                 elif board[user_move] != ' ':
91                     print("That spot is already taken. Try another one.")
92                 else:
93                     break
94             except ValueError:
95                 print("Please enter a valid number.")
96
97         board[user_move] = 'X'
98         print_board()
99
100        if check_winner(board, 'X'):
101            print("You win!")
102            break
103        if is_full(board):
104            print("Draw!")
105            break
106
107        computer_move = best_move()
108        board[computer_move] = 'O'
109        print("AI moved:")
110        print_board()
111
```

```
112         if check_winner(board, '0'):  
113             print("AI wins!")  
114             break  
115         if is_full(board):  
116             print("Draw!")  
117             break  
118  
119  
120 play_game()
```

Sample Output:

```
Tic-Tac-Toe (AI using Alpha-Beta Pruning)  
| |  
-----  
| |  
-----  
| |  
Your move (0-8): 0  
X| |  
-----  
| |  
-----  
| |  
AI moved:  
X| |  
-----  
|0|  
-----  
| |  
Your move (0-8): 2  
X| |X  
-----  
|0|  
-----  
| |  
AI moved:  
X|0|X  
-----  
|0|  
-----  
| |  
Your move (0-8): 4  
That spot is already taken. Try another one.  
Your move (0-8): 6  
X|0|X  
-----  
|0|
```

```
-----  
X| |  
AI moved:  
X|O|X  
-----  
O|O|  
-----  
X| |  
Your move (0-8): 5  
X|O|X  
-----  
O|O|X  
-----  
X| |  
AI moved:  
X|O|X  
-----  
O|O|X  
-----  
X|O|  
AI wins!
```

Result: Alpha-Beta Pruning was successfully implemented for Tic-Tac-Toe, producing an optimal computer player while avoiding unnecessary branches during the search.

Date: 21/07/2026

Experiment - 06

Cryptarithmic Puzzle

Aim: To implement a constraint satisfaction approach for solving a cryptarithmic addition puzzle by assigning unique digits to letters.

Algorithm:

- Step 1 : Start the program and read a cryptarithmic equation from the user.
- Step 2 : Remove spaces, convert the equation to uppercase, and separate both sides of the equation.
- Step 3 : Collect the words participating in the addition and the result word.
- Step 4 : Find all unique letters and verify that their count does not exceed ten.
- Step 5 : Identify the first letter of every word because a leading letter cannot represent zero.
- Step 6 : Generate permutations of digits for all unique letters.
- Step 7 : Create a letter-to-digit mapping for each permutation.
- Step 8 : Reject mappings containing repeated digits or assigning zero to a leading letter.
- Step 9 : Convert every word into its corresponding numeric value.
- Step 10 : Check whether the sum of the left-side numbers equals the result, then print the first valid mapping and end the program.

Code:

```
1 from itertools import permutations
2
3
4 def solve(puzzle):
5     puzzle = puzzle.replace(" ", "").upper()
6
7     if "=" not in puzzle or "+" not in puzzle:
8         print("Enter a valid addition puzzle.")
9         return
10
11     left_side, result_word = puzzle.split("=")
12     addend_words = left_side.split("+")
13     words = addend_words + [result_word]
14
15     letters = sorted(set("".join(words)))
16     if len(letters) > 10:
17         print("Too many unique letters. A maximum of 10 is allowed.")
18         return
```

```
19 leading_letters = {word[0] for word in words}
20
21
22 for permutation in permutations("0123456789", len(letters)):
23     mapping = dict(zip(letters, permutation))
24
25     if any(mapping[letter] == "0" for letter in leading_letters):
26         continue
27
28     numbers = [
29         int("".join(mapping[letter] for letter in word))
30         for word in words ]
31
32     if sum(numbers[:-1]) == numbers[-1]:
33         print("Solution found!")
34         print("Mapping:", mapping)
35         print(
36             "Numeric form:",
37             " + ".join(map(str, numbers[:-1])),
38             "=",
39             numbers[-1])
40         return
41
42     print("No solution found.")
43
44
45 puzzle = input(
46     "Enter a cryptarithmic puzzle "
47     "(e.g. SEND + MORE = MONEY): ")
48 solve(puzzle)
```

Sample Output:

```
Enter a cryptarithmic puzzle (e.g. SEND + MORE = MONEY): SEND + MORE =
MONEY
Solution found!
Mapping: {'D': '7', 'E': '5', 'M': '1', 'N': '6', 'O': '0', 'R': '8', 'S':
'9', 'Y': '2'}
Numeric form: 9567 + 1085 = 10652
```

Result: The cryptarithmic puzzle was successfully solved by treating letter-to-digit assignment as a constraint satisfaction problem.